

Running in Production with Docker and Fly.io

Contents

1. Overview	2
2. Preventing the Missing <code>AGE_KEY</code> Error	3
2.1. Injecting the key at container start	3
2.2. Using an untracked env file for local development	3
2.3. Configuring the key as a Fly.io secret	4
3. Preventing the Database <code>localhost</code> Error	5
4. Recommended Long-Term Setup	6
5. Keeping Production Values Out of Local Files	7
6. Summary	8

patrick toca v0.8.1, October 7th, 2026

Creation Date: October 7th, 2026

Author: patrick toca

Email: pmjtoca@gmail.com

This document describes how to run the MyWebApp container in production and local environments, focusing on the two most common runtime failures: a missing `AGE_KEY` and an unreachable PostgreSQL host. Both failures come from missing runtime configuration, not from the image itself.

1. Overview

The container image is self-contained. Every failure observed so far has been caused by configuration that must be supplied at **runtime**, not build time:

1. A missing `AGE_KEY`, which the application requires to decrypt secrets.
2. A `DB_HOST` value that points to `localhost`, which inside a container refers to the container itself, not the host running PostgreSQL.

This document covers how to supply both values correctly for local Docker, Docker Compose, and Fly.io.

2. Preventing the Missing AGE_KEY Error

The application fails fast when AGE_KEY is absent. This is intentional: a missing decryption key must never be silently ignored.

2.1. Injecting the key at container start

Pass the key directly when starting the container. Read it from your local age key file, stripping comment lines:

```
docker run --rm \
  -e AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)" \
  ...
mywebapp:debug
```



Passing secrets on the command line exposes them to shell history and to any user who can run `ps`. This form is acceptable only for a quick local test. Use an env file or a platform secret for anything persistent.

2.2. Using an untracked env file for local development

Create a local environment file containing the required runtime values:

```
AGE_KEY=AGE-SECRET-KEY-...
DB_HOST=host.docker.internal
DB_PORT=5432
DB_USER=pnjtoca
DB_NAME=MyWebApp
DB_SSLMODE=disable
```

Then start the container with that file:

```
docker run --rm \
  --env-file .env.docker.local \
  --add-host host.docker.internal:host-gateway \
  -p 8080:8080 \
  mywebapp:debug
```

Add the env file to `.gitignore` so it is never committed:

```
.env.docker.local
```



Never put `AGE_KEY` in the Dockerfile, in image layers, in Git, or in a committed `.env` file. Any of those locations turns a runtime secret into a permanent, widely distributed artifact.

2.3. Configuring the key as a Fly.io secret

On Fly.io, store the key as a platform secret rather than an environment file:

```
fly secrets set AGE_KEY="$(grep -v '^#' ~/.config/sops/age/keys.txt)"
```

Fly injects the secret into the running machine at start time, so it never appears in the image or in a repository.

3. Preventing the Database `localhost` Error

Inside a container, `localhost` refers to the container's own network namespace. PostgreSQL is almost never there. Set `DB_HOST` according to where the database actually runs:

Where PostgreSQL runs	Value for <code>DB_HOST</code>
On the host machine	<code>host.docker.internal</code>
In Docker Compose	The database service name, for example <code>db</code>
On Fly.io	The private or managed database hostname
Anywhere else	Its reachable DNS name or IP address

Your local `docker run` command should therefore include:

```
-e DB_HOST=host.docker.internal
```

In addition to setting `DB_HOST`, ensure that PostgreSQL itself accepts connections from Docker:

- Its `listen_addresses` setting must include the interface the container reaches it on.
- Its `pg_hba.conf` must contain a rule permitting the Docker network or the container's source address.
- Any host firewall must allow traffic from the Docker bridge network to the PostgreSQL port.

4. Recommended Long-Term Setup

For local development, replace ad-hoc `docker run` invocations with a Compose file. This makes the runtime configuration explicit and reproducible.

Create `compose.yaml` in the repository root:

```
services:
  app:
    build: .
    ports:
      - "8080:8080"
    env_file:
      - .env.docker.local
    extra_hosts:
      - "host.docker.internal:host-gateway"
```

Run it with:

```
docker compose up --pull always
```

The `extra_hosts` entry is what makes `host.docker.internal` resolve on Linux, where Docker does not provide that name by default.

5. Keeping Production Values Out of Local Files

Local env files are a convenience for development only. For production:

- Keep every secret in Fly secrets (or your platform's equivalent secret store).
- Keep non-secret configuration in the deployment configuration, not in a committed env file.
- Keep the application's fail-fast behavior. The process should continue to exit immediately when the age key or the database configuration is missing, because a server that starts without them is a server that will fail later, under load, in a harder-to-diagnose way.

6. Summary

1. Supply `AGE_KEY` at runtime, through an untracked env file locally or a platform secret in production.
2. Set `DB_HOST` to the real location of PostgreSQL, never to `localhost` from inside a container.
3. Use `compose.yaml` for local runs, and Fly secrets for production.
4. Preserve fail-fast behavior on missing configuration.